🎯 LAB 1: Node.js Fundamentals & Async Programming

Weeks 1-2 | Learn How Node.js Really Works!

Instructor: Mohamed Amine Marzouk
Difficulty: Beginner
Time Estimate: 6-8 hours
Outcome: AA1 (Node.js Architecture & Async)

📍 WHERE WE ARE IN THE COURSE

You're starting the **foundation** of Node.js. These first two weeks are CRITICAL because:

- ☑ You'll understand how Node.js is different from regular JavaScript
- ☑ You'll learn the "secret sauce" that makes Node.js fast
- ☑ You'll master async/await (the skill ALL Node.js developers need)
- ☑ After this lab, everything else will make sense!

🎬 WHY THIS MATTERS

Think about your phone charging:

```
❌ OLD WAY (BLOCKING - Bad):
  1. Plug phone in
  2. WAIT for it to charge (can't do anything else!)
  3. 2 hours later... finally ready
  4. Now charge tablet
  5. WAIT for tablet (still can't do anything)
  6. Total time: 4 hours

☑ NEW WAY (NON-BLOCKING - Good):
  1. Plug phone in (charge in background)
  2. Plug tablet in (charge in background)
  3. While they charge, READ A BOOK or WATCH TV!
  4. When phone is done, charger notifies you
  5. When tablet is done, charger notifies you
  6. Total time: 2 hours (both at same time!)
```

This is what Node.js does! It doesn't wait. It handles multiple things at once. That's why Instagram, Netflix, and Uber use Node.js.

📦 WHAT YOU'LL BUILD

By the end of this lab, you'll create an **Event Logger System** that:

- ☑ Understands the **Event Loop** (Node.js engine)
- ☑ Uses **callbacks** (old way)
- ☑ Uses **Promises** (better way)
- ☑ Uses **Async/Await** (best way!)
- ☑ Runs code in **parallel** and **sequential** patterns
- ☑ Handles errors properly in async code

Real-world use: When users upload events to your app, you need to:

- Save event to database (takes time)
- Send confirmation email (takes time)
- Update user notifications (takes time)
- Do ALL OF THESE at the same time!

Without async: User waits 10 seconds
With async: User waits 0.5 seconds! 🚀

☑ PRE-LAB CHECKLIST

Before you start, verify:

- ☐ Node.js 25+ installed (`node --version`)
- ☐ You can run: `node hello.js` (from pre-course)
- ☐ You understand: variables, functions, objects, arrays
- ☐ You've read the pre-course "Synchronous vs Asynchronous" section
- ☐ You have a code editor (VS Code)
- ☐ You have a folder for this course (like `NodeJS_Course/Lab1`)

Not ready? Go back to **PRE_COURSE_PREREQUISITES.md** Section 4.

TASK 1: UNDERSTAND THE EVENT LOOP

What is the Event Loop?

Imagine you're at a restaurant:

```
👨 Manager (Event Loop)
└─ Keeps checking: "Is there work to do?"

Steps the Manager Follows:
1. Check line of waiting customers (Synchronous code)
2. Send them to available chef (Execute)
3. While chef cooks, check next customer
4. Check if any food is done (Promises, setTimeout)
5. Deliver finished food
```

```
6. Go back to step 1
7. NEVER SITS IDLE!
```

The manager is the **Event Loop**. They're constantly checking for work and assigning it.

In Node.js:

- **Synchronous code** = Customers in the line
- **Async code** = Food cooking in the kitchen
- **Event Loop** = Manager organizing everything
- **Callback** = "When food is done, bring it to table"

TASK 1.1: See the Event Loop in Action

Step 1: Create a file called `Lab1_EventLoop.js`

```
# In your Lab1 folder, create the file:
cd NodeJS_Course/Lab1
touch Lab1_EventLoop.js
```

Step 2: Copy this code into the file

```
// Lab1_EventLoop.js
// This code shows you the ORDER that JavaScript executes things

console.log('🟢 STEP 1: I run first (synchronous code)');

// setTimeout is ASYNCHRONOUS (takes time)
setTimeout(() => {
  console.log('🟡 STEP 3: I run after 0ms delay (macro task)');
}, 0);  // Even with 0ms delay, still waits!

// Promise is ASYNCHRONOUS (but different type)
Promise.resolve()
  .then(() => {
    console.log('🟠 STEP 2: I run before setTimeout (micro task)');
  });

console.log('🟢 STEP 4: I run second (synchronous code)');

// EXPECTED OUTPUT:
// 🟢 STEP 1: I run first (synchronous code)
// 🟢 STEP 4: I run second (synchronous code)
// 🟠 STEP 2: I run before setTimeout (micro task)
// 🟡 STEP 3: I run after 0ms delay (macro task)
```

Step 3: Run it!

```
node Lab1_EventLoop.js
```

You should see:

```
🟢 STEP 1: I run first (synchronous code)
🟢 STEP 4: I run second (synchronous code)
🟡 STEP 2: I run before setTimeout (micro task)
🔴 STEP 3: I run after 0ms delay (macro task)
```

Step 4: Understand the Order

Why this order?

1. **Synchronous code runs first** → `console.log` at top and bottom
2. **Then micro tasks** → Promises run before setTimeout
3. **Then macro tasks** → setTimeout runs last

This is the **Event Loop** determining order!

Step 5: Modify and Experiment!

Try these changes:

```
// Change setTimeout delay to 1000ms (1 second)
setTimeout(() => {
  console.log('This takes 1 second!');
}, 1000);

// Add more Promises
Promise.resolve()
  .then(() => console.log('Promise 1'))
  .then(() => console.log('Promise 2'))
  .then(() => console.log('Promise 3'));

// What order do you expect?
// Try it and see!
```

☑ CHECKPOINT 1

Before moving on, you should be able to:

- ☐ Understand what "asynchronous" means (not blocking)
- ☐ Explain the Event Loop (as restaurant manager)

- ☐ Know the order of execution (sync → microtasks → macrotasks)
- ☐ Predict the output of async code

If not, review the code and run it again!

TASK 2: CALLBACK PATTERN (OLD WAY)

What is a Callback?

A callback is a function you pass to another function that gets called later.

Real-world analogy:

```
"Hey Uber driver, pick me up when you arrive"
  ↓
That function that runs when they arrive = callback
```

TASK 2.1: Callbacks with File Reading

Step 1: Create `Lab1_Callbacks.js`

```
touch Lab1_Callbacks.js
```

Step 2: Write this code

```
// Lab1_Callbacks.js
// Callbacks are how Node.js used to handle async operations

const fs = require('fs');

console.log('Starting to read file...');

// THIS IS A CALLBACK:
// The second parameter is a function that runs LATER
fs.readFile('sample.txt', 'utf8', (error, data) => {
  // This function RUNS WHEN THE FILE IS COMPLETELY READ

  if (error) {
    console.log('❌ ERROR:', error.message);
  } else {
    console.log('✅ File content:');
    console.log(data);
  }
});
```

```
console.log('File reading started (still waiting for it to finish)...');

// QUESTION: What prints first?
// "Starting..." → "File reading started" → "File content"
// (The file reading happens in the background!)
```

Step 3: Create a sample file

```
echo "Hello from file! This is sample data." > sample.txt
```

Step 4: Run it!

```
node Lab1_Callbacks.js
```

Output:

```
Starting to read file...
File reading started (still waiting for it to finish)...
☑ File content:
Hello from file! This is sample data.
```

Notice:

The "File reading started" line prints BEFORE the file content!
That's because the file read happens in the **background**. That's async!

TASK 2.2: Error Handling in Callbacks

Update your `Lab1_Callbacks.js` to handle errors:

```
// Lab1_Callbacks.js - IMPROVED with error handling

const fs = require('fs');

function readEventFile(filename, callback) {
  // This function takes a CALLBACK parameter
  fs.readFile(filename, 'utf8', (error, data) => {
    if (error) {
      // Something went wrong
      callback(error, null);  // Pass error to callback
    } else {
      try {
```

```
        // Try to parse JSON
        const eventData = JSON.parse(data);
        callback(null, eventData);  // Pass data to callback
      } catch (parseError) {
        // JSON parsing failed
        callback(parseError, null);
      }
    }
  });
}

// USING THE CALLBACK:
readEventFile('sample.txt', (error, data) => {
  if (error) {
    console.log('❌ Error:', error.message);
  } else {
    console.log('✅ Data:', data);
  }
});
```

TASK 2.3: The "Callback Hell" Problem

This is what happens when you nest callbacks (NOT GOOD!):

```
// ❌ CALLBACK HELL - Don't write like this!
readFile('file1.txt', (error1, data1) => {
  if (!error1) {
    processData(data1, (error2, result1) => {
      if (!error2) {
        saveResult(result1, (error3, result2) => {
          if (!error3) {
            notifyUser(result2, (error4, result3) => {
              // Too many nested brackets!
              // Hard to read!
              // Easy to make mistakes!
              console.log(result3);
            });
          }
        });
      }
    });
  }
});

// This is why Promises and Async/Await were invented!
```

✅ CHECKPOINT 2

You should now understand:

- ☐ What a callback is (function passed to another function)
- ☐ Why callbacks are asynchronous (they run later)
- ☐ How to handle errors in callbacks
- ☐ Why "callback hell" is bad (hard to read and maintain)

TASK 3: PROMISE PATTERN (BETTER WAY)

What is a Promise?

A Promise is like ordering food at a restaurant:

```
1. Place order (create Promise)
2. Waiter says "Your order will be ready in 15 minutes"
3. You get a receipt (the Promise)
4. After 15 minutes:
   - ☑ Food arrives (Promise RESOLVED)
   - ❌ OR they're out of ingredients (Promise REJECTED)
```

Promises give you a cleaner way to handle asynchronous operations.

TASK 3.1: Create a Promise

Create `Lab1_Promises.js`

```javascript
// Lab1_Promises.js
// Promises are the modern way to handle async operations

const fs = require('fs').promises;  // Note: .promises version!

console.log('=== READING FILE WITH PROMISES ===\n');

// Create a Promise that reads a file
function readEventFile(filename) {
  return fs.readFile(filename, 'utf8')
    // .then() runs AFTER file is read
    .then(data => {
      try {
        return JSON.parse(data);  // Try to parse as JSON
      } catch (err) {
        throw err;  // If parsing fails, throw error
      }
    })
    // .catch() runs if ANYTHING goes wrong
    .catch(error => {
```

```
      console.log('❌ Error reading file:', error.message);
      throw error;  // Re-throw so calling code knows there was error
    });
}

// USING THE PROMISE:
readEventFile('sample.txt')
  .then(data => {
    console.log('✅ Successfully read file!');
    console.log('Data:', data);
  })
  .catch(error => {
    console.log('❌ Final error handler:', error.message);
  })
  .finally(() => {
    console.log('✅ Operation completed (either success or failure)');
  });

console.log('File reading started (promise created)...');
```

Run it:

```
node Lab1_Promises.js
```

TASK 3.2: Promise Chaining

Chaining means doing operations one after another:

```
// Lab1_Promises_Chaining.js
const fs = require('fs').promises;

// Read event1.json, then event2.json, then event3.json
// ONE AFTER ANOTHER

fs.readFile('event1.json', 'utf8')
  .then(data1 => {
    console.log('✅ Read event1:', data1);
    return fs.readFile('event2.json', 'utf8');  // Chain to next operation
  })
  .then(data2 => {
    console.log('✅ Read event2:', data2);
    return fs.readFile('event3.json', 'utf8');  // Chain again
  })
  .then(data3 => {
    console.log('✅ Read event3:', data3);
    console.log('✅ All files read in sequence!');
  })
  .catch(error => {
```

```
    console.log('❌ Error:', error.message);
  });

console.log('Starting to read files...');
```

Much better than callback hell! Much cleaner code!

☑ CHECKPOINT 3

You should understand:

- ☐ What a Promise is (pending, resolved, rejected)
- ☐ .then() runs when Promise resolves
- ☐ .catch() runs when Promise rejects
- ☐ .finally() runs always (success or failure)
- ☐ Chaining Promises (.then().then().then())

TASK 4: ASYNC/AWAIT PATTERN (BEST WAY! ★)

What is Async/Await?

Async/Await is the **modern, cleanest way** to write asynchronous code.

It makes async code look like **synchronous code** (but it's not!).

```
// ❌ OLD WAY (Promises - still okay):
function getEvent() {
  return readFile('event.json')
    .then(data => {
      return parseData(data);
    })
    .then(event => {
      return saveEvent(event);
    });
}

// ☑ NEW WAY (Async/Await - looks like normal code!):
async function getEvent() {
  const data = await readFile('event.json');
  const event = await parseData(data);
  await saveEvent(event);
  return event;
}
```

Which looks easier to read? The second one!

TASK 4.1: Your First Async Function

Create `Lab1_AsyncAwait.js`

```
// Lab1_AsyncAwait.js
// THIS IS THE WAY TO WRITE ASYNC CODE IN 2026!

const fs = require('fs').promises;

// NOTE: Function must be marked as "async"
async function readEventFile(filename) {
  try {
    // AWAIT means "wait for this operation to finish"
    const data = await fs.readFile(filename, 'utf8');
    console.log('✅ File read successfully!');

    // Parse the data
    const event = JSON.parse(data);
    return event;

  } catch (error) {
    // If ANYTHING goes wrong, catch it here
    console.log('❌ Error:', error.message);
    throw error;
  }
}

// CALLING AN ASYNC FUNCTION:
// You still need "await" when you call it!

async function main() {
  try {
    console.log('Starting to read event...\n');

    const event = await readEventFile('sample.txt');

    console.log('Event data:', event);
    console.log('\n✅ Complete!');

  } catch (error) {
    console.log('❌ Something went wrong:', error.message);
  }
}

// RUN THE MAIN FUNCTION:
main();
```

Run it:

```
node Lab1_AsyncAwait.js
```

TASK 4.2: Understanding Async/Await Syntax

Key rules:

```
// RULE 1: Function must have "async" keyword
async function doSomething() {
  // Can use "await" inside
}

// RULE 2: "await" pauses execution until Promise completes
const result = await someAsyncOperation();

// RULE 3: Error handling uses try/catch (like regular code!)
async function example() {
  try {
    const result = await operation();
  } catch (error) {
    // Handle error here
  }
}

// RULE 4: "async" functions always return a Promise
const promise = myAsyncFunction();  // This is a Promise!
const result = await myAsyncFunction();  // This waits for Promise
```

TASK 4.3: Handling Multiple Events

Create `Lab1_Async_MultipleEvents.js`

```
// Lab1_Async_MultipleEvents.js
// Reading MULTIPLE files

const fs = require('fs').promises;

// Create some test event files first:
async function createTestFiles() {
  const event1 = JSON.stringify({
    id: 1,
    title: 'Tech Conference',
    date: '2024-03-15'
  });

  const event2 = JSON.stringify({
    id: 2,
    title: 'Web Workshop',
    date: '2024-04-20'
```

```
  });

  await fs.writeFile('event1.json', event1);
  await fs.writeFile('event2.json', event2);
  console.log('✅ Test files created\n');
}

// Read multiple files SEQUENTIALLY (one after another)
async function readEventsSequential() {
  try {
    console.log('📖 SEQUENTIAL: Reading one by one...');

    const event1 = await fs.readFile('event1.json', 'utf8');
    console.log('✅ Event 1:', event1);

    const event2 = await fs.readFile('event2.json', 'utf8');
    console.log('✅ Event 2:', event2);

    console.log('Done!\n');
  } catch (error) {
    console.log('❌ Error:', error.message);
  }
}

// Read multiple files PARALLEL (all at the same time!)
async function readEventsParallel() {
  try {
    console.log('⚡ PARALLEL: Reading all at once...');

    // Promise.all() runs ALL promises at the same time!
    const [event1, event2] = await Promise.all([
      fs.readFile('event1.json', 'utf8'),
      fs.readFile('event2.json', 'utf8')
    ]);

    console.log('✅ Event 1:', event1);
    console.log('✅ Event 2:', event2);

    console.log('Done!\n');
  } catch (error) {
    console.log('❌ Error:', error.message);
  }
}

// RUN EVERYTHING:
async function main() {
  await createTestFiles();
  await readEventsSequential();
  await readEventsParallel();
}

main();
```

Run it:

```
node Lab1_Async_MultipleEvents.js
```

TASK 4.4: Parallel vs Sequential Performance

Let's see the SPEED difference!

Create `Lab1_Performance.js`

```
// Lab1_Performance.js
// Shows that PARALLEL is faster than SEQUENTIAL

const fs = require('fs').promises;
const path = require('path');

// Create test files
async function setupTest() {
  for (let i = 1; i <= 3; i++) {
    const data = JSON.stringify({
      id: i,
      title: `Event ${i}`,
      date: `2024-0${i}-15`
    });
    await fs.writeFile(`event${i}.json`, data);
  }
}

// SEQUENTIAL: Read one after another
async function testSequential() {
  console.time('⏱️  SEQUENTIAL');

  const event1 = await fs.readFile('event1.json', 'utf8');
  const event2 = await fs.readFile('event2.json', 'utf8');
  const event3 = await fs.readFile('event3.json', 'utf8');

  console.timeEnd('⏱️  SEQUENTIAL');
  console.log('✅ Read 3 files sequentially\n');
}

// PARALLEL: Read all at the same time
async function testParallel() {
  console.time('⚡ PARALLEL');

  const [event1, event2, event3] = await Promise.all([
    fs.readFile('event1.json', 'utf8'),
    fs.readFile('event2.json', 'utf8'),
    fs.readFile('event3.json', 'utf8')
  ]);
```

```
  console.timeEnd('⚡ PARALLEL');
  console.log('✅ Read 3 files in parallel\n');
}

// RUN TEST:
async function main() {
  await setupTest();

  console.log('🚀 PERFORMANCE COMPARISON\n');

  await testSequential();
  await testParallel();

  console.log('⚡ Parallel is MUCH faster!');
}

main();
```

Run it:

```
node Lab1_Performance.js
```

SEQUENTIAL waits for each file (slower!)
PARALLEL reads all at once (faster!)

✅ CHECKPOINT 4

You should now:

- ☐ Understand async/await syntax
- ☐ Know when to use "await"
- ☐ Use try/catch for error handling
- ☐ Know the difference between sequential and parallel
- ☐ Understand when to use Promise.all()

FINAL CHALLENGE: CREATE EVENT LOGGER SYSTEM

Your Mission

Build a complete **Event Logger System** that:

1. ✅ Creates event files (JSON format)
2. ✅ Reads events (async/await)
3. ✅ Processes events (filtering, sorting)

4. ☑ Logs to file
5. ☑ Handles all errors gracefully

CREATE: `Lab1_EventLogger.js`

```
// Lab1_EventLogger.js
// Complete Event Logger System using async/await!

const fs = require('fs').promises;
const path = require('path');

// STEP 1: Create sample events
async function createSampleEvents() {
  const events = [
    {
      id: 1,
      title: 'Tech Conference 2024',
      date: '2024-03-15',
      capacity: 500,
      registrations: 250
    },
    {
      id: 2,
      title: 'Web Dev Workshop',
      date: '2024-04-20',
      capacity: 50,
      registrations: 48
    },
    {
      id: 3,
      title: 'JavaScript Masterclass',
      date: '2024-05-10',
      capacity: 100,
      registrations: 95
    }
  ];

  for (const event of events) {
    const filename = `event_${event.id}.json`;
    await fs.writeFile(filename, JSON.stringify(event, null, 2));
  }

  console.log('☑ Created 3 sample event files\n');
}

// STEP 2: Read all events (PARALLEL!)
async function readAllEvents() {
  try {
    console.log('📖 Reading all event files...');

    const files = await fs.readdir('.');
```

```
    const eventFiles = files.filter(f => f.startsWith('event_') &&
f.endsWith('.json'));

    // Read all events IN PARALLEL
    const eventPromises = eventFiles.map(file =>
      fs.readFile(file, 'utf8').then(data => JSON.parse(data))
    );

    const events = await Promise.all(eventPromises);
    console.log(`✅ Read ${events.length} events\n`);

    return events;
  } catch (error) {
    console.log('❌ Error reading events:', error.message);
    throw error;
  }
}

// STEP 3: Process and analyze events
async function analyzeEvents(events) {
  console.log('📊 ANALYZING EVENTS:\n');

  // Sort by date
  events.sort((a, b) => new Date(a.date) - new Date(b.date));

  for (const event of events) {
    const occupancy = (event.registrations / event.capacity * 100).toFixed(1);
    console.log(`📍 ${event.title}`);
    console.log(`   Date: ${event.date}`);
    console.log(`   Occupancy: ${event.registrations}/${event.capacity}
(${occupancy}%)`);
    console.log();
  }
}

// STEP 4: Save analysis to log file
async function saveAnalysisLog(events) {
  try {
    console.log('💾 Saving analysis to log...');

    let log = `EVENT ANALYSIS LOG\nCreated: ${new Date().toISOString()}\n\n`;

    for (const event of events) {
      const occupancy = (event.registrations / event.capacity * 100).toFixed(1);
      log += `${event.title} - ${occupancy}% full\n`;
    }

    await fs.writeFile('analysis.log', log);
    console.log('✅ Log saved to analysis.log\n');
  } catch (error) {
    console.log('❌ Error saving log:', error.message);
  }
}
```

```
// STEP 5: Run everything!
async function main() {
  try {
    console.log('🚀 EVENT LOGGER SYSTEM\n');
    console.log('='.repeat(40) + '\n');

    // Create test events
    await createSampleEvents();

    // Read all events
    const events = await readAllEvents();

    // Analyze events
    await analyzeEvents(events);

    // Save results
    await saveAnalysisLog(events);

    console.log('✅ ALL TASKS COMPLETED!');

  } catch (error) {
    console.log('❌ FATAL ERROR:', error.message);
  }
}

// START THE PROGRAM:
main();
```

Run it:

```
node Lab1_EventLogger.js
```

🎯 WHAT YOU'VE LEARNED

- ✅ **Event Loop** - How Node.js executes code
- ✅ **Synchronous vs Asynchronous** - Blocking vs non-blocking
- ✅ **Callbacks** - Old way to handle async
- ✅ **Promises** - Better way with .then() and .catch()
- ✅ **Async/Await** - Best modern way (looks like sync code!)
- ✅ **Sequential vs Parallel** - When to use each
- ✅ **Error Handling** - try/catch patterns
- ✅ **Real Application** - Built an event logger system

✅ LAB 1 DELIVERABLES

Submit these files:

1. ☐ `Lab1_EventLoop.js` - Event loop demo
2. ☐ `Lab1_Callbacks.js` - Callback pattern
3. ☐ `Lab1_Promises.js` - Promise pattern
4. ☐ `Lab1_AsyncAwait.js` - Async/await pattern
5. ☐ `Lab1_Performance.js` - Sequential vs parallel comparison
6. ☐ `Lab1_EventLogger.js` - Complete event logger system

Grading:

- Each file runs without errors: 20 points
- Demonstrates understanding: 60 points
- Code is well-commented: 20 points

Total: 100 points

🎁 BONUS CHALLENGES

Want to go deeper? Try these:

Bonus 1: Error Scenarios

```
// Test what happens when file doesn't exist
const event = await fs.readFile('nonexistent.json', 'utf8');
// Your error handling should catch this gracefully
```

Bonus 2: Timeout Simulation

```
// Create a function that takes time (like database query)
function simulateDatabaseQuery(delayMs) {
  return new Promise(resolve => {
    setTimeout(() => resolve('Database result'), delayMs);
  });
}

// Use it with async/await
const result = await simulateDatabaseQuery(2000);
```

Bonus 3: Complex Workflow

```
// Create a workflow:
// 1. Read event from file
// 2. Wait 2 seconds (simulate processing)
// 3. Save result to new file
// 4. Log completion
// All with proper error handling!
```

🆘 TROUBLESHOOTING

"await is only valid in async function"

- Make sure your function has `async` keyword

"Cannot find module"

- Check your file paths
- Make sure files are in the same directory

"JSON.parse error"

- Check that JSON file is valid
- Use JSON.stringify to create files correctly

Promise stays pending forever

- Make sure you're using `await` to wait for it
- Check that the async operation is actually completing

📚 RESOURCES

- Node.js async guide: https://nodejs.org/en/docs/guides/blocking-vs-non-blocking/
- MDN Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- MDN Async/Await: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Async_await

---

✨ FINAL THOUGHTS

Lab 1 is **the most important lab** in the course.

If you master async/await here, everything else (databases, APIs, authentication) will be easy!

Don't rush. Take time to understand the Event Loop. Run the code multiple times. Modify it. Break it. Fix it.

When you truly understand async/await, you'll understand Node.js.

Congratulations on completing Lab 1! 🎉

You now understand how Node.js really works!

Next Lab: Modules & Package Management

Instructor: Mohamed Amine Marzouk
Email: med.amine.mzk@horizon-tech.tn

Last Updated: January 25, 2026
Status: Ready for students